Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу «Операционные системы»

Группа: М8О-209БВ-25

Студент: Алимов Р.Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 18.10.26

Москва, 2026

Постановка задачи

Вариант 2. Параллельная быстрая сортировка (QuickSort). Реализовать последовательную и многопоточную сортировку массива целых чисел. Для параллельной версии использовать POSIX Threads и ограничение на число потоков, задаваемое аргументом командной строки. Сравнить время выполнения последовательной и параллельной версий для различных значений ограничения, вычислить ускорение и эффективность.

Общий метод и алгоритм решения

Программа написана на C и состоит из main.c, sort.c, sort.h и Makefile. В main.c создаются два массива целых чисел одинакового размера. Первый заполняется значениями rand() после вызова srand(42), второй получает копию данных с помощью memcpy(). Поэтому оба алгоритма обрабатывают одинаковые исходные значения.

Последовательная функция quick_sort() выполняет быструю сортировку с разбиением массива функцией partition(). Опорным элементом выбирается значение из середины текущего участка; разбиение выполняется двумя индексами, движущимися навстречу друг другу (схема Хоара). После разбиения сортируются обе полученные части.

В parallel_sort() после разбиения при достаточном размере участка и наличии свободного места в установленном лимите создаётся поток pthread_create() для сортировки левой части. Правую часть обрабатывает текущий поток. После этого вызывается pthread_join(). Если создание нового потока не предусмотрено или не удалось, обе части сортируются в текущем потоке. Новые потоки не создаются для участков, у которых right - left < 10000.

Глобальные счётчики active_threads и peak_threads защищены спин-блокировкой на atomic_flag. Счётчик active_threads изначально равен 1, так как включает главный поток. peak_threads хранит наибольшее зарегистрированное число занятых потоковых слотов. Критические секции содержат только операции со счётчиками; сама сортировка выполняется без блокировки.

Время измеряется функцией get_time() с помощью clock_gettime(CLOCK_MONOTONIC) и выводится в миллисекундах. После выполнения обеих сортировок результаты сравниваются функцией memcmp(); при совпадении выводится сообщение Sort: OK.

Использованные средства

- pthread_create() — создаёт поток для сортировки части массива.
- pthread_join() — дожидается завершения созданного потока перед возвратом из рекурсивной функции.
- atomic_flag_test_and_set() и atomic_flag_clear() — обеспечивают захват и освобождение спин-мьютекса.
- clock_gettime(CLOCK_MONOTONIC) — позволяет измерять прошедшее время.
- malloc(), rand(), memcpy(), memcmp() — выделение памяти, генерация входных данных, копирование и сравнение массивов.

Код программы

sort.h

```
#ifndef SORT_H
#define SORT_H

#define _POSIX_C_SOURCE 200809L
#include <pthread.h>
#include <time.h>

extern int max_threads;
extern int peak_threads;
```

```
void quick_sort(int *a, int left, int right);
void parallel_sort(int *a, int left, int right);
double get_time(void);

#endif
```

sort.c

```
#include "sort.h"
#include <stdatomic.h>

#define MIN_PART 10000

typedef struct {
   int *array;
   int left;
   int right;
} Task;

static atomic_flag mutex = ATOMIC_FLAG_INIT;

void mutex_lock(void) {
   while (atomic_flag_test_and_set(&mutex)) {
      // Ждем
   }
}

void mutex_unlock(void) {
   atomic_flag_clear(&mutex);
}

int max_threads;
int active_threads = 1;
int peak_threads = 1;

static int partition(int *a, int left, int right) {
   int pivot = a[(left + right) / 2];
   int i = left - 1;
   int j = right + 1;

   while (1) {
      do { i++; } while (a[i] < pivot);
      do { j--; } while (a[j] > pivot);

      if (i >= j) return j;

      int tmp = a[i];
      a[i] = a[j];
      a[j] = tmp;
   }
}

void quick_sort(int *a, int left, int right) {
   if (left >= right) return;

   int p = partition(a, left, right);
   quick_sort(a, left, p);
   quick_sort(a, p + 1, right);
}

static void *thread_sort(void *arg) {
   Task *task = (Task *)arg;
   parallel_sort(task->array, task->left, task->right);
   return NULL;
}

void parallel_sort(int *a, int left, int right) {
   if (left >= right) return;

   int p = partition(a, left, right);
   int create = 0;

   if (max_threads > 1 && right - left >= MIN_PART) {
      mutex_lock();
      if (active_threads < max_threads) {
         active_threads++;
         create = 1;
      }
```

```
        mutex_unlock();
    }

    if (create) {
        pthread_t thread;
        Task task = {a, left, p};

        if (pthread_create(&thread, NULL, thread_sort, &task) == 0) {
            mutex_lock();
            if (active_threads > peak_threads)
                peak_threads = active_threads;
            mutex_unlock();

            parallel_sort(a, p + 1, right);
            pthread_join(thread, NULL);

            mutex_lock();
            active_threads--;
            mutex_unlock();
            return;
        }

        mutex_lock();
        active_threads--;
        mutex_unlock();
    }

    parallel_sort(a, left, p);
    parallel_sort(a, p + 1, right);
}

double get_time(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1000.0 + t.tv_nsec / 1000000.0;
}
```

main.c

```
#include "sort.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc != 3) {
        printf("Usage: %s <array_size> <max_threads>\n", argv[0]);
        return 1;
    }

    int n = atoi(argv[1]);
    max_threads = atoi(argv[2]);

    int *a = malloc((size_t)n * sizeof(int));
    int *b = malloc((size_t)n * sizeof(int));
    if (a == NULL || b == NULL) {
        printf("Memory allocation error\n");
        free(a);
        free(b);
        return 1;
    }

    srand(42);
    for (int i = 0; i < n; i++)
        a[i] = rand();
    memcpy(b, a, (size_t)n * sizeof(int));

    double start = get_time();
    quick_sort(a, 0, n - 1);
    double time = get_time() - start;

    printf("PID: %ld\n", (long)getpid());
    printf("Array size: %d, thread limit: %d\n", n, max_threads);
    fflush(stdout);

    start = get_time();
```

```
    parallel_sort(b, 0, n - 1);
    double parallel_time = get_time() - start;

    if (memcmp(a, b, (size_t)n * sizeof(int)) != 0) {
        printf("Sorting error\n");
        free(a);
        free(b);
        return 1;
    }

    printf("Sequential: %.3f ms\n", time);
    printf("Parallel:   %.3f ms\n", parallel_time);
    printf("Speedup:    %.3f\n", time / parallel_time);
    printf("Efficiency: %.3f\n", time / parallel_time / max_threads);
    printf("Peak threads: %d\n", peak_threads);
    printf("Sort: OK\n");

    free(a);
    free(b);
    return 0;
}
```

Makefile

```
all: main.out

main.out: main.c sort.c sort.h
	gcc -std=c11 -O2 -Wall -Wextra -pedantic -pthread -o main.out main.c sort.c

clean:
	rm -f main.out

.PHONY: all clean
```

Протокол работы программы

Программа собрана командой make. В Makefile используются параметры компиляции -std=c11 -O2 -Wall -Wextra -pedantic -pthread. Измерения выполнены на массиве из 5 000 000 целых чисел. Для каждого ограничения количества потоков проведено по пять запусков. По имеющимся данным в Ubuntu доступны 16 логических процессоров (команда nproc).

Пример запуска при 16 потоках

```
$ ./main.out 5000000 16
PID: 102812
Array size: 5000000, thread limit: 16
Sequential: 289.076 ms
Parallel:   105.901 ms
Speedup:    2.730
Efficiency: 0.171
Peak threads: 16
Sort: OK
```

Во всех 40 запусках на предоставленных скриншотах программа выводила Sort: OK. Во всех показанных запусках значение Peak threads было равно заданному пределу.

Результаты отдельных запусков

Таблица 1 — измерения в миллисекундах. В каждой ячейке указаны последовательное / параллельное время одного запуска.

p	Запуск 1	Запуск 2	Запуск 3	Запуск 4	Запуск 5
1	294.085 / 309.992	287.704 / 280.950	353.205 / 375.526	284.653 / 291.598	286.904 / 302.667
2	286.666 / 274.494	281.294 / 284.430	295.887 / 289.017	291.157 / 276.649	291.318 / 274.395
4	301.069 / 169.846	291.437 / 189.764	286.910 / 180.196	289.963 / 193.781	289.723 / 190.499
8	294.632 / 171.401	297.264 / 156.184	296.437 / 140.638	290.785 / 129.656	282.486 / 149.484
16	289.076 / 105.901	288.257 / 91.457	287.450 / 97.754	284.106 / 100.852	299.332 / 104.471
64	284.512 / 111.569	326.032 / 105.710	293.207 / 91.112	290.136 / 103.943	291.825 / 112.137
128	293.795 / 107.929	295.004 / 115.819	288.363 / 107.151	290.516 / 101.805	287.970 / 125.145
256	282.421 / 127.668	288.222 / 126.525	287.989 / 127.619	298.729 / 97.018	284.446 / 138.635

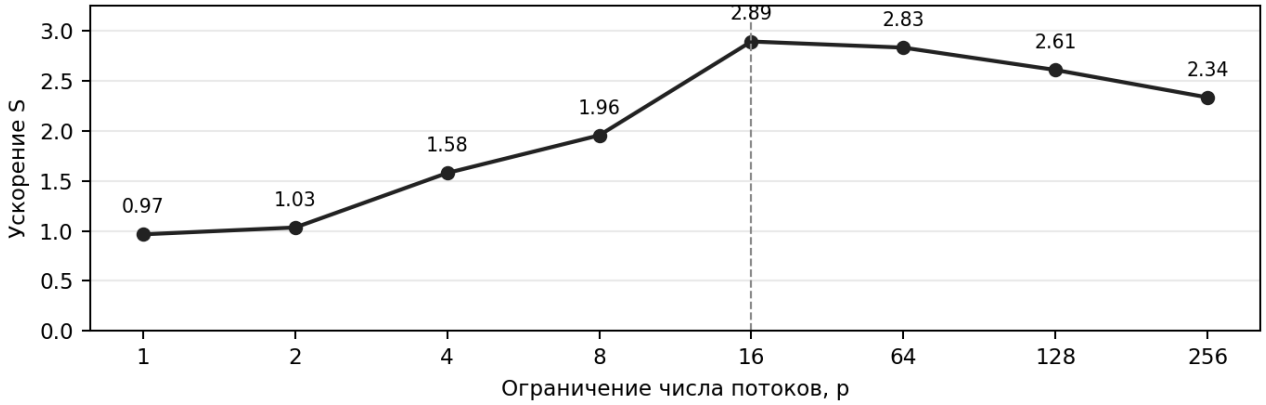
Средние показатели

Средние значения времени вычислены по пяти запускам для каждого p. Ускорение и эффективность рассчитаны по формулам S = Tпосл. / Tпар. и E = S / p, где p — заданное максимальное число потоков.

Таблица 2 — результаты эксперимента (среднее время в миллисекундах).

Потоки p	Последовательно	Параллельно	Ускорение S	Эффективность E
1	301.310	312.147	0.965	0.965
2	289.264	279.797	1.034	0.517
4	291.820	184.817	1.579	0.395
8	292.321	149.473	1.956	0.244
16	**289.644**	**100.087**	**2.894**	**0.181**
64	297.142	104.894	2.833	0.044
128	291.130	111.570	2.609	0.020
256	288.361	123.493	2.335	0.009

Рисунок 1 — ускорение параллельной сортировки в зависимости от ограничения числа потоков.



Анализ результатов

При p = 1 среднее ускорение составило 0,965, то есть параллельная функция без создания дополнительных потоков оказалась немного медленнее последовательной. С ростом числа потоков от 2 до 16 среднее время параллельной сортировки уменьшилось с 279,797 до 100,087 мс. Максимальное среднее ускорение в проведённых экспериментах — 2,894 при p = 16.

При p = 64, 128 и 256 среднее параллельное время выросло до 104,894; 111,570 и 123,493 мс соответственно. Увеличение числа потоков сверх числа логических процессоров не привело к дальнейшему ускорению. Возможные причины — расходы на создание потоков, конкуренция за процессорное время и накладные расходы синхронизации.

Показатель Peak threads вычислен внутри программы по счётчику занятых потоковых слотов; представленные измерения не содержат отдельного наблюдения потоков средствами ОС, например командой ps -T.

Вывод

В работе реализованы последовательная и параллельная версии QuickSort на C с POSIX Threads. Защита счётчика потоков выполнена собственным spinlock на atomic_flag, ожидание дочерних потоков — через pthread_join(). Все 40 запусков завершились с Sort: OK. При сортировке 5 000 000 элементов максимальное среднее ускорение составило 2,894 при 16 потоках; при 64, 128 и 256 потоках скорость уменьшилась из-за дополнительных затрат параллельного выполнения.